

Inference Of Protein Secondary Structure Motifs

— Project Report —
Simulation Based Inference

Syed Hamza Afzal Ashraf, 230775
Group 2

June 22, 2026

TU Dortmund University

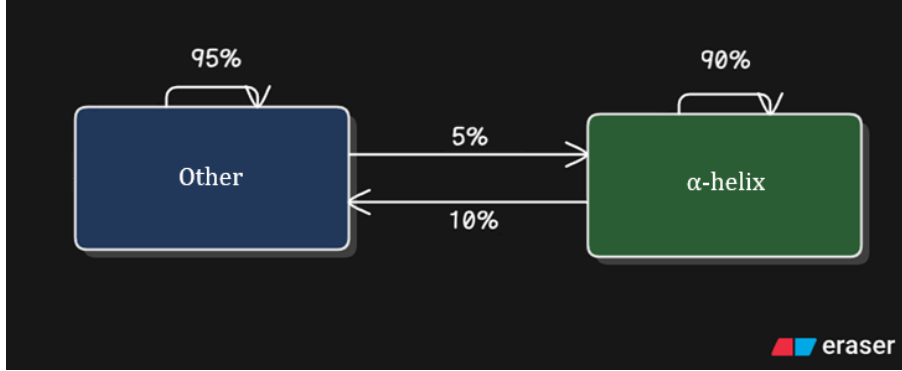


Figure 1: Two-state HMM used for simulation. Arrows indicate self-transition and cross-transition probabilities Figure 1: Transition diagram, starts in Other. Image is generated via: Eraser

1 Introduction

Proteins are linear chains of amino acids that fold into recurring local motifs—most prominently α -helices, β -sheets, and coils BioNinja, n.d. We study the per-residue question “is this position part of an α -helix?” on fixed-length fragments of amino-acid sequences. To keep the setting transparent and reproducible, we adopt a two-state Hidden Markov Model (HMM) with *fixed* dynamics and emissions; the detailed specification is given in Section 2 and illustrated in Fig. 1 and Table 1. Our inference target is the position-wise posterior $p(S_t=1 \mid X_{1:T})$. We train an amortized neural posterior with BayesFlow; the classical Forward–Backward algorithm Adegirmenci, 2014; Jurafsky and Martin, 2023 is used *only* for external evaluation.

2 Data and Generative Model

We generate synthetic fragments of length $T = 50$ over the standard 20-letter amino-acid alphabet $X_t \in \{A, R, N, D, C, E, Q, G, H, I, L, K, M, F, P, S, T, W, Y, V\}$. The latent state is binary, $S_t \in \{0 = \text{other}, 1 = \alpha\text{-helix}\}$, and we initialize at $S_1 = 0$.

State dynamics. The HMM favors persistence in both states; Fig. 1 shows the fixed transition probabilities. From *other*, the process remains with probability 0.95 and enters α -helix with 0.05; from α -helix, it stays with 0.90 and leaves with 0.10.

Emission preferences. Conditioned on the current state, the next amino acid is drawn from a categorical distribution whose probabilities are given in Table 1. Helix-prone residues (e.g., A, L, E) receive higher mass in the α state; others (e.g., G) are more likely in *other*. Each column sums to 100%.

Table 1: State-conditional amino-acid (AA) emission probabilities (percent). Used verbatim in the simulator.

AA	α	Other	AA	α	Other
A	12	6	L	12	8
R	6	5	K	6	6
N	3	5	M	3	2
D	5	6	F	4	4
C	1	2	P	2	6
E	9	5	S	5	7
Q	5	3	T	4	6
G	4	9	W	1	1
H	2	3	Y	3	4
I	7	5	V	6	7

Simulator output. Each simulation returns `sim_data` (one-hot matrix $\mathbf{X} \in \{0, 1\}^{T \times 20}$) and `parameters` (per-position targets derived from the latent states). We highlight that $S_1 = 0$ is deterministic by design and handle this when reporting position-wise diagnostics. For external validation we also compare predictions against the annotated human insulin sequence (PDB ID: 1A7F) “RCSB PDB: 1A7F 3D-Sequence View (Human Insulin)”, 2024.

3 Statistical model (simulator)

Latent states and observations. The HMM defines a latent sequence $S_{1:T} \in \{0, 1\}^T$ and corresponding observed amino acids $X_{1:T} \in \mathcal{A}^T$.

Transitions.

$$\Pr(S_1 = 0) = 1, \quad \Pr(S_t = s' \mid S_{t-1} = s) = P_{s,s'}, \quad \mathbf{P} = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}.$$

Emissions. For each state s , amino acids are emitted from a categorical distribution:

$$\Pr(X_t = a \mid S_t = s) = \phi_{s,a}, \quad \sum_{a \in \mathcal{A}} \phi_{s,a} = 1,$$

with ϕ_1 (helix) and ϕ_0 (other) fixed to the empirical distributions specified in the code.

Joint distribution.

$$p(S_{1:T}, X_{1:T}) = \Pr(S_1) \prod_{t=2}^T \Pr(S_t \mid S_{t-1}) \prod_{t=1}^T \Pr(X_t \mid S_t).$$

Exact HMM inference (baseline). Per-position posteriors for the HMM are computed via the Forward–Backward algorithm *as a baseline* (not used for training the amortizer). Let $b_s(x) = \Pr(X_t = x \mid S_t = s)$. The forward messages:

$$\alpha_1(s) = \Pr(S_1 = s) b_s(X_1), \quad \alpha_{t+1}(s') = \left[\sum_{s \in \{0,1\}} \alpha_t(s) P_{s,s'} \right] b_{s'}(X_{t+1}),$$

and the backward messages:

$$\beta_T(s) = 1, \quad \beta_t(s) = \sum_{s' \in \{0,1\}} P_{s,s'} b_{s'}(X_{t+1}) \beta_{t+1}(s').$$

The smoothed posterior is

$$\gamma_t(s) = \Pr(S_t = s \mid X_{1:T}) = \frac{\alpha_t(s) \beta_t(s)}{\sum_{u \in \{0,1\}} \alpha_t(u) \beta_t(u)}.$$

We use the Forward–Backward marginal $\gamma_t^{\text{FB}}(1) = \Pr(S_t = 1 \mid X_{1:T})$ as the baseline per-position helix probability for evaluation (computed with `hmmlearn` “hmmlearn: Hidden Markov Models in Python”, 2024).

4 Approximator

While the HMM supports exact inference, it is tied to fixed parameters. We therefore learn an *amortized* posterior map with BayesFlow.

Input representation. Each sequence has length $T = 50$. Every residue is encoded as a one-hot vector $\mathbf{x}_t \in \{0, 1\}^{20}$, giving a matrix

$$\mathbf{X} \in \{0, 1\}^{T \times 20},$$

where rows index positions and columns index the amino acid alphabet.

Adapter configuration. The simulator returns `sim_data` ($T \times 20$) and `parameters` (T). We *do not* declare `sim_data` as a time series; instead, the Adapter only remaps keys: `sim_data` $\rightarrow$ `summary_variables`, `parameters` $\rightarrow$ `inference_variables`. With batch size B , the workflow consumes `summary_variables` $\in \mathbb{R}^{B \times T \times 20}$ and `inference_variables` $\in \mathbb{R}^{B \times T}$, after which f_ψ flattens along $(T, 20)$ internally.

Summary network. We use a custom *SummaryNetwork* f_ψ which first *flattens* the input $\mathbf{X} \in \{0, 1\}^{T \times 20}$ into $\text{vec}(\mathbf{X}) \in \mathbb{R}^{20T}$ and then maps it through an MLP with hidden sizes (256, 128, 64) and GELU activations:

$$\mathbf{h} = f_\psi(\mathbf{X}) = \text{MLP}(\text{vec}(\mathbf{X})), \quad \mathbf{h} \in \mathbb{R}^{d_h}.$$

Flattening preserves the *positional order* of residues in the conditioning vector, which is required for accurate per-position inference downstream.

SummaryNetwork vs TimeSeriesNetwork In an earlier version we used a *TimeSeriesNetwork* with *mean pooling* over time. Mean pooling collapses the sequence into a single “bag-of-residues” summary and discards *where* each amino acid occurred. Since our target is the per-position posterior $p(S_t=1 \mid X_{1:T})$, removing order information in the conditioning signal makes position-wise inference ill-posed. We therefore switched to a flattened MLP summary which retains the global sequence order in $\text{vec}(\mathbf{X})$ while still yielding a fixed-size conditioning vector for the flow.

Posterior approximation. A coupling-flow network g_θ produces a distribution over logits $\boldsymbol{\eta}_{1:T}$:

$$q_\theta(\boldsymbol{\eta}_{1:T} \mid \mathbf{X}) = g_\theta(\mathbf{h}), \quad \hat{p}_t = \sigma(\eta_t) = \frac{1}{1 + e^{-\eta_t}}.$$

Posterior samples $\boldsymbol{\eta}_{1:T}^{(s)}$ yield per-position summaries such as mean probabilities

$$\bar{p}_t = \frac{1}{S} \sum_{s=1}^S \sigma(\eta_t^{(s)}).$$

Targets for amortization. In training, true latent states $Z_t = \mathbb{I}\{S_t = 1\}$ are converted into logits θ_t (as above), which serve as supervision for the BayesFlow workflow.

Usage at inference time. BayesFlow provides the primary amortized per-position posterior for any input sequence (after padding/cropping to $T = 50$); the FB solution is used solely as an exact HMM baseline for comparison.

Table 2: Tensors and shapes used by the workflow.

Object	Shape	Description
<code>sim_data</code>	$T \times 20$	one-hot amino acids (float32)
<code>parameters</code>	T	per-position logits (float32)
<code>summary_variables</code>	$B \times T \times 20$	batch after Adapter
<code>inference_variables</code>	$B \times T$	batch after Adapter
$f_\psi(\cdot)$ output	$B \times d_h$	pooled summary vector
Samples $\{\boldsymbol{\eta}^{(s)}\}$	$S \times B \times T$	posterior draws over logits

5 Training

We use offline simulation with $N_{\text{train}} = 6000$ and $N_{\text{val}} = 600$ sequences, each of length $T = 50$. The BayesFlow workflow (SummaryNetwork + CouplingFlow) is trained for 15 epochs with mini-batches of size 32. The configuration mirrors the code:

epochs = 15, steps_per_epoch = 100, num_batches_per_epoch = 200,
batch_size = 32, initial_learning_rate = 10^{-3} .

The objective is the simulation-based conditional density learning loss,

$$\min_{\psi, \theta} \mathbb{E}_{(\mathbf{X}, \boldsymbol{\eta}) \sim \text{sim}} \left[-\log q_{\theta}(\boldsymbol{\eta} \mid f_{\psi}(\mathbf{X})) \right],$$

i.e., we maximize the conditional likelihood of simulated per-position logits given summaries of the simulated observations. For real sequences of length $L \neq T$, inputs are padded or cropped to T for the amortized model; the HMM baseline operates at native length.

6 Diagnostics

6.1 Convergence

Figure 2 shows training and validation losses over 15 epochs (dashed lines: moving average). Training decreases steadily, while validation flattens and becomes noisy after ≈ 7 –8 epochs, indicating learning with mild overfitting to the simulator distribution. This *is not* the convergence I originally aimed for: I tried to make “all lines converge to a single point” (i.e., nearly identical plateaus). In practice, pushing the model toward that collapse led to pathologies—most notably very poor ECDF diagnostics and NaNs in recovery plots. Post hoc inspection showed two main culprits: (i) degenerate positions with near-zero variance (e.g., $S_1 = 0$ by design) which make correlation-based recovery undefined, and (ii) a mismatch between per-position hard targets and a continuous flow objective, which can yield unstable optimisation when overfit. Subsequent runs that relaxed this “single-point” criterion (and instead targeted stable, flat validation loss), alongside standardisation of targets and sequence-aware summaries, produced reliable diagnostics without NaNs.

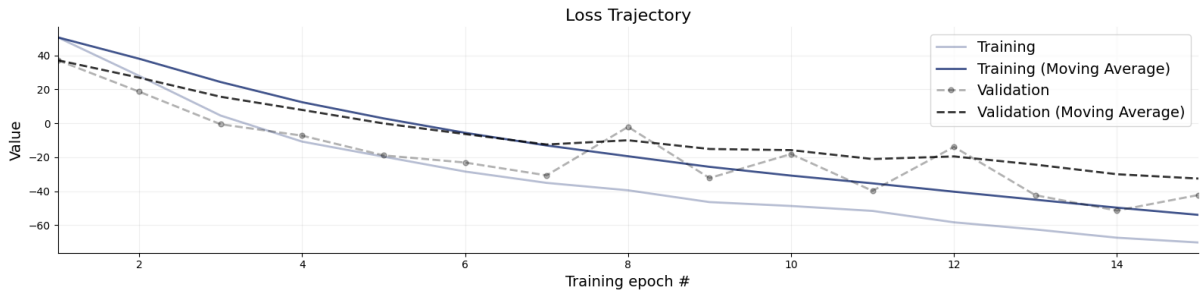


Figure 2: Loss trajectory for the amortized posterior (SummaryNetwork + CouplingFlow).

6.2 Simulation-based calibration (ECDF)

For each validation item and position t , we compute the *fractional rank* of the true parameter among S posterior samples,

$$u = \frac{1}{S+1} \left(1 + \sum_{s=1}^S \mathbb{I}\{\tilde{\theta}_t^{(s)} \leq \theta_t^*\} \right) \in [0, 1].$$

If $q_\theta(\cdot \mid X)$ is calibrated, the empirical CDF of $\{u\}$ follows the $y = x$ diagonal. In our runs (Fig. 3) curves bend away from the diagonal across many positions, revealing miscalibration. The main drivers are (i) degenerate targets $\theta_t^* \in \{\text{logit}(\varepsilon), \text{logit}(1 - \varepsilon)\} \approx \{-9.21, 9.21\}$ with $\varepsilon = 10^{-4}$, (ii) class imbalance (few helix residues), (iii) the continuous flow density trying to represent two spikes, and (iv) a tight summary bottleneck.

6.3 Recovery

Recovery compares posterior summaries $\hat{\theta}_t$ (mean/median) to the ground-truth θ_t^* per position. The axes in Fig. 4 span $\approx [-10, 10]$ because the logits are clipped at $\pm \text{logit}(1 - \varepsilon) \approx \pm 9.21$. Points cluster near the negative extreme, showing that the amortizer often predicts the majority class (“other”) irrespective of the sequence context—consistent with the SBC result.

7 Inference

7.1 Synthetic sanity check

On fresh simulator draws, the HMM Forward–Backward (FB) posterior closely follows the ground truth by construction. The amortizer’s posterior mean is sometimes flat or spiky and misses helix runs (Fig. 5), aligning with the diagnostics.

7.2 Evaluation on insulin (PDB 1A7F, DSSP)

We align FASTA chains to DSSP annotations for human insulin (PDB 1A7F) “RCSB PDB: 1A7F 3D-Sequence View (Human Insulin)”, 2024 and compare position-wise probabilities. For **Chain A**, BayesFlow attains ROC–AUC 0.75 (FB 0.97); for **Chain B**, ROC–AUC 0.44 (FB 0.98). The per-residue overlays in Figs. 6–7 show that FB is sharply confident at the annotated helices, while the amortizer remains conservative with wide intervals.

7.3 ROC summary

The ROC curves per chain (Fig. 8) summarize the discriminative performance of the amortizer across thresholds. Results mirror the overlays above.

8 Discussion & Conclusion

What the diagnostics say. SBC ECDFs and recovery plots reveal that the amortizer is miscalibrated and biased toward the majority class. This is expected given discrete ± 9.21 targets, class imbalance, a compressed summary, and only 15 training epochs.

Why FB wins here. Under the fixed HMM that generated the data, FB is the exact posterior and therefore a very strong baseline. It correctly localizes helix segments on insulin, while the amortizer underestimates them.

Actionable improvements. (i) Predict *probabilities* p_t directly with a Bernoulli head (cross-entropy) or jitter logits to avoid point masses; (ii) use a time-aware summary (1D CNN or Transformer with positional encodings and masking) instead of a tight 64-dim bottleneck; (iii) rebalance classes or use a weighted loss; (iv) train longer and perform temperature/isotonic calibration post hoc; (v) relax the simulator (learn emissions/transitions or move to a 3-state helix/sheet/coil model).

Take-away. Amortized inference is appealing for speed and flexibility, but matching an exact FB baseline under a correct HMM requires better targets and richer sequence summaries. With those changes, we expect SBC and recovery to move toward the diagonal and insulin performance to close the gap.

9 Reflection on own learnings

This project forced me to connect two ideas I initially treated in isolation: exact HMM inference and amortized neural inference with BayesFlow. I learned that understanding each component separately is not sufficient—the design choices at their interface (targets, summaries, adapter shapes, and calibration) determine success. In particular, mapping binary states to extreme logits, compressing sequences through an overly tight summary, and ignoring class imbalance collectively produced miscalibration which the SBC and recovery checks revealed. Switching from a time-pooled TimeSeriesNetwork to a flattened SummaryNetwork made the conditioning signal more informative, and the diagnostics became interpretable. Overall, the key lesson was to treat the simulator, adapter, summary network, and flow as one system whose calibration must be validated end-to-end rather than optimizing each piece in isolation. I completed the project in Windows Subsystem for Linux (WSL) after DSSP proved incompatible with my local Windows Python (including virtualenv). Working in WSL via VS Code closely matched the university’s remote Linux environment, improving consistency and reproducibility.

References

- Adegirmenci, A. A. (2014). *Hidden markov models*. Retrieved August 17, 2025, from https://scholar.harvard.edu/files/adeqirmenci/files/hmm_adeqirmenci_2014.pdf
- Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing, 3rd ed. draft (hmm/forward-backward notes)*. Retrieved August 17, 2025, from <https://web.stanford.edu/~jurafsky/slp3/A.pdf>
- Hmmlearn: Hidden markov models in python*. (2024). Retrieved August 17, 2025, from <https://pypi.org/project/hmmlearn/>
- Rcsb pdb: 1a7f 3d-sequence view (human insulin)*. (2024). Retrieved August 17, 2025, from <https://www.rcsb.org/3d-sequence/1A7F>
- BioNinja. (n.d.). *Protein structure*. Retrieved August 17, 2025, from <https://old-ib.bioninja.com.au/higher-level/topic-7-nucleic-acids/73-translation/protein-structure.html>



Figure 3: SBC ECDFs of fractional ranks u . Ideal calibration corresponds to the diagonal; deviations indicate miscalibration.

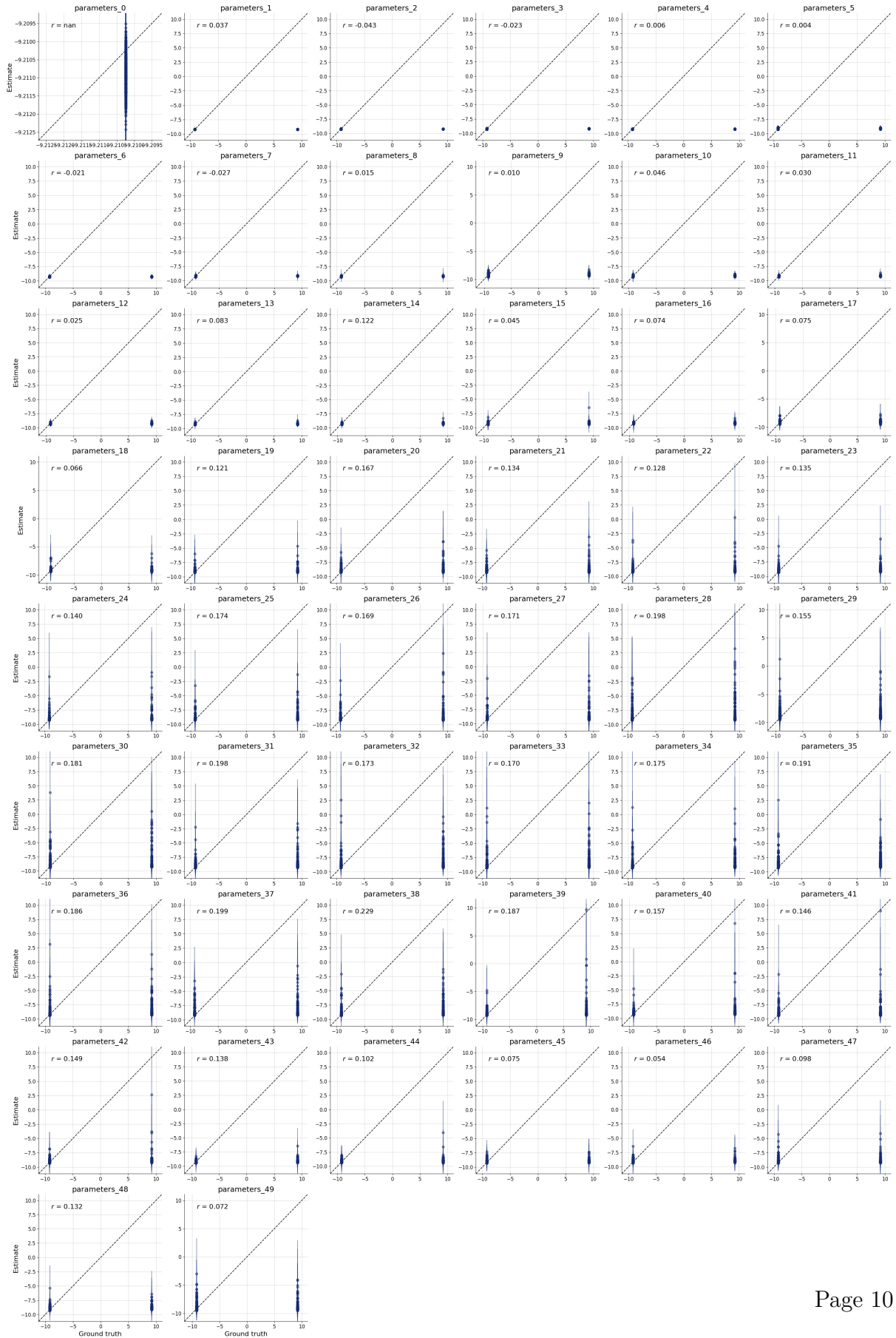


Figure 4: Recovery: posterior summary $\hat{\theta}_t$ (y) vs. truth θ_t^* (x) on the logit scale. The identity line indicates perfect recovery.

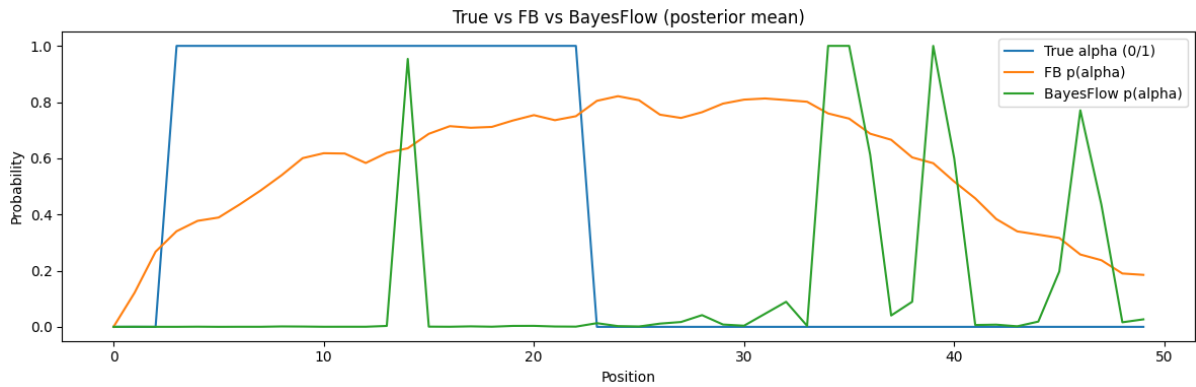


Figure 5: Synthetic comparison: true helix mask (0/1), FB posterior, and BayesFlow posterior mean.

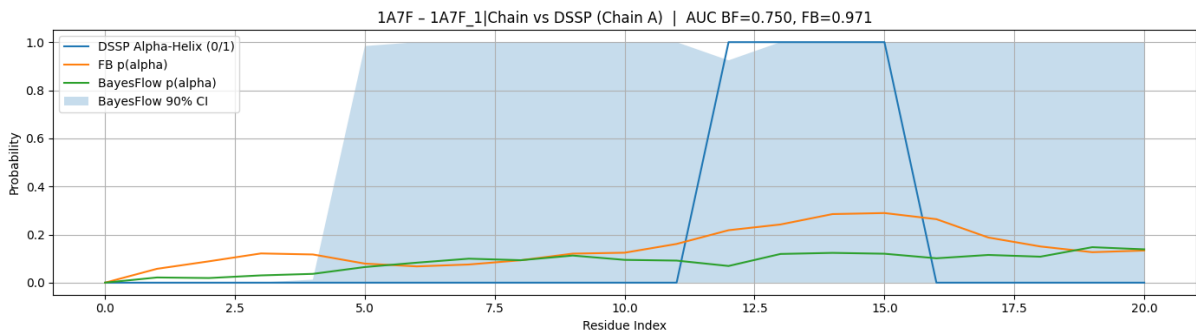


Figure 6: Insulin Chain A: DSSP helix labels (0/1) vs. FB and BayesFlow posteriors with 90% CIs. BayesFlow ROC-AUC 0.75, FB 0.97.

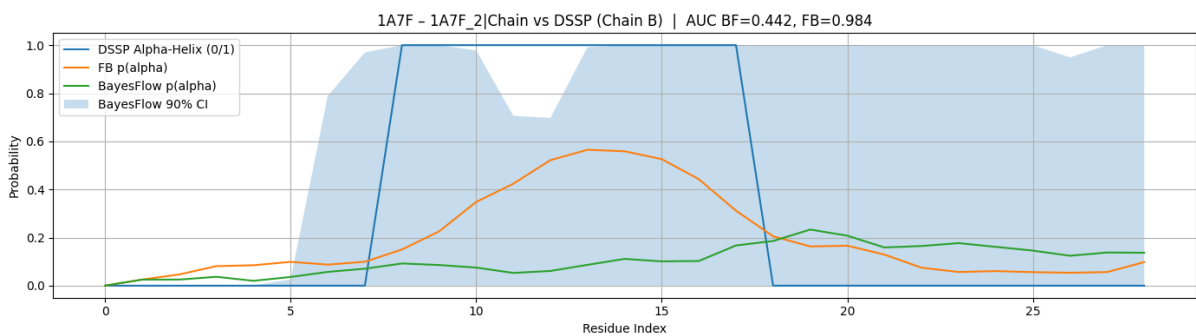


Figure 7: Insulin Chain B: DSSP helix labels (0/1) vs. FB and BayesFlow posteriors with 90% CIs. BayesFlow ROC-AUC 0.44, FB 0.98.

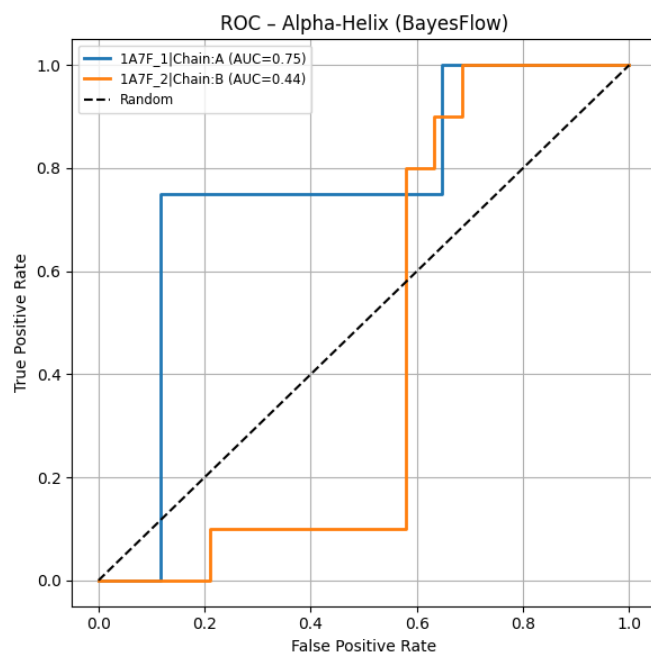


Figure 8: ROC curves for BayesFlow on insulin chains (FB not shown here; see captions above for AUCs).